

Machine Learning

Reinforcement Learning

Marco Mussi

Credits to Alberto Maria Metelli and Francesco Provò

Politecnico di Milano

Reinforcement Learning For Prediction

Possible Options for Prediction

When we want to perform prediction and we do not know the environment dynamics or modeling the environment is too complex:

- Monte Carlo (first and every visit)

$$V(s_t) \leftarrow V(s_t) + \alpha(v_t - V(s_t))$$

- Temporal Difference

$$V(s_t) \leftarrow V(s_t) + \alpha(r_{t+1} + \gamma V(s_{t+1}) - V(s_t))$$

- $TD(\lambda)$ (eligibility traces)

$$V(s_t) \leftarrow V(s_t) + \alpha(v_t^\lambda - V(s_t))$$

$$\text{with } v_t^\lambda = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} v_t^{(n)}$$

Exercise 9.5

Evaluate the value for the MDP with states $\mathcal{S} = \{A, B, C\}$ (C is terminal), actions $\mathcal{A} = \{h, r\}$ given the policy π and the following trajectories:

$$(A, h, 3) \rightarrow (B, r, 2) \rightarrow (B, h, 1) \rightarrow (C)$$

$$(A, h, 2) \rightarrow (A, h, 1) \rightarrow (C)$$

$$(B, r, 1) \rightarrow (A, h, 1) \rightarrow (C)$$

- ❶ Can you tell without computing anything if by resorting to MC with every-visit and first-visit approach you will have different results?
- ❷ Compute the value functions with the two aforementioned MC methods (assume $\gamma = 1$).
- ❸ Assume to consider a discount factor $\gamma = 1$ and compute the values by resorting to TD? Assume to start from zero values for each state and $\alpha = 0.1$

Reinforcement Learning For Control

Possible Options for Control

- Monte Carlo Control:
 - Policy evaluation: Monte Carlo Estimation
 - Policy improvement: ε -greedy
- SARSA:
 - Policy evaluation: Temporal Difference Estimation
 - Policy improvement: ε -greedy
- Q-learning: empirical version of Value Iteration

SARSA

The SARSA algorithm iterates between:

- An environment step, using the transition model
- A policy improvement step, with the ϵ -greedy policy
- An evaluation step, with the TD update of the Q function:

$$Q(s, a) \leftarrow Q(s, a) + \alpha(r + \gamma Q(s', a') - Q(s, a))$$

Q-learning

The Q-learning algorithm iterates over:

- An environment step, using the transition model
- A policy improvement step, with the ϵ -greedy policy
- An update with the Bellman optimality equation:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left(r + \gamma \max_{\tilde{a} \in \mathcal{A}} Q(s', \tilde{a}) - Q(s, a) \right)$$

On-policy vs Off-policy

- With Q-learning I could have had a dataset to use for learning (off-policy learning)
- With SARSA I need to execute the ε -greedy policy at each time point (on-policy learning)